

Introduction to Optimization

Kendra Burbank

- Applied and computational real analysis
 - How the abstract stuff you learned can be applied to real world problems
 - How you can compute their solutions!
- Bridge undergraduate real analysis and graduate courses on optimization

Notation (for today and for the rest of the course)

- $\mathbb{R}^{m \times n}$ is the set of all $m \times n$ matrices
- $\mathbb{S}^n$ is the set of all $n \times n$ symmetric matrices,
- $\mathbb{S}_+^n$ is the set of all $n \times n$ symmetric positive semidefinite matrices.

What is optimization?

In an optimization problem, we try to **minimize** an objective function f subject to a set of constraints.

These constraints can be **equality** constraints (e.g. $g = 0$) or **inequality** constraints (e.g. $h \leq 0$).

In general, we can write the problem as:

$$\text{minimize} \quad f(x_1, \dots, x_n)$$

subject to:

$$\begin{cases} g_1(x_1, \dots, x_n) = 0 \\ \vdots \\ g_m(x_1, \dots, x_n) = 0 \\ h_1(x_1, \dots, x_n) \leq 0 \\ \vdots \\ h_p(x_1, \dots, x_n) \leq 0 \end{cases}$$

Note that we may have a number (e.g. m) of equality constraints and a number (e.g. p) of inequality constraints!

We wrote our objective function f as a function of many variables:

$$f(x_1, \dots, x_n)$$

In this course, sometimes we'll work with *univariate* functions, like $f(x)$.

Often, we'll work with *multivariate* functions, like $f(x_1, \dots, x_n)$.

We'll also work with *matrix-argument functions*, which we'll write as $f(X)$.

When we have multivariate functions, it's often useful to think of the arguments as a vector.

This lets us efficiently work with many variables at once.

For example, we can write our multivariate function as $f(\mathbf{x})$, where $\mathbf{x} = (x_1, \dots, x_n)$.

Then the general optimization problem can be written as:

$$\begin{array}{ll} \text{minimize} & f(\mathbf{x}) \\ \text{subject to} & g_i(\mathbf{x}) = 0, \quad i = 1, \dots, m \\ & h_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, p \end{array}$$

The set of variables we're optimizing over may be related to each other in a way that makes it natural to think of them as a matrix.

For example, they may be the elements of a covariance matrix or a network's connection weights.

In these cases, our objective function f looks like this:

$$f(X) = f \left(\begin{bmatrix} x_{11} & \cdots & x_{1n} \\ \vdots & \ddots & \vdots \\ x_{m1} & \cdots & x_{mn} \end{bmatrix} \right)$$

Our general optimization problem becomes:

$$\begin{array}{ll} \text{minimize} & f(X) \\ \text{subject to} & g_i(X) = 0, \quad i = 1, \dots, m \\ & h_i(X) \leq 0, \quad i = 1, \dots, p \end{array}$$

**Every problem is an optimization
problem**

Reframing problems as optimization problems

Many difficult-to-solve problems can be reframed as optimization problems.

Basic idea: create an objective function **whose optimal value is the solution to our original problem.**

Example: Proving Fermat's last theorem

Theorem

$x^n + y^n = z^n$ has no integer solutions for $n \geq 3$

This was a famous problem that went unsolved for centuries.

In 1994, Andrew Wiles proved Fermat's Last Theorem in a proof that involved first reframing the theorem as an optimization problem.

What could this look like?

Theorem

$x^n + y^n = z^n$ has no integer solutions for $n \geq 3$

First, we can say that if there is a solution, then $x^n + y^n = z^n$ so $x^n + y^n - z^n = 0$.

So we might say something like “if the minimum value of $x^n + y^n - z^n$ is 0, then there is a solution to the original problem”.

Except that this might be *negative*.

Maybe let's square things, say something like “if the minimum value of $(x^n + y^n - z^n)^2$ is 0, then there is a solution to the original problem”.

But we haven't restricted ourselves to integers!

Theorem

$x^n + y^n = z^n$ has no integer solutions for $n \geq 3$

Clever idea: if x is an integer, then $\sin(\pi x) = 0$.

Same goes for y and z .

So the following function will be 0 if and only if $x^n + y^n = z^n$ and x, y, z are integers:

$$f(x, y, z) = (x^n + y^n - z^n)^2 + \sin(\pi x) + \sin(\pi y) + \sin(\pi z)$$

Hmm. I guess this function would be 0 also if $x, y, \text{ and } z$ were all 0 as well... we don't want that... so we can say they need to be ≥ 1 .

Finally, we can see our reformulated problem:

$$\begin{array}{ll}\text{minimize} & f(x, y, z) = (x^n + y^n - z^n)^2 + \sin(\pi x) + \sin(\pi y) + \sin(\pi z) \\ \text{subject to} & x \geq 1, y \geq 1, z \geq 1, n \geq 3\end{array}$$

If the minimum value of this function is 0, then there is a solution to the original problem and Fermat's Last Theorem is false.

Example: Partial differential equations

Suppose we have a partial differential equation (PDE) that describes the behavior of a physical system.

For example, the *heat equation* describes how heat flows through a region over time.

If $f(\mathbf{x}, t)$ is the temperature at point $\mathbf{x}$ and time t , then:

$$\frac{\partial f}{\partial t} = \frac{\partial^2 f}{\partial x_1^2} + \cdots + \frac{\partial^2 f}{\partial x_n^2} =$$

The right-hand side is the **Laplacian** of f , which we can write as $\nabla^2 f$ or Δf .

We often want to find the behavior of the system in some region of space when we are given the values of the function on the boundary of that region.

For example, how does heat flow through a metal plate, when we hold the temperature constant in a certain pattern on the edges of the plate?

We typically represent the region of interest as Ω , and its boundary as $\partial\Omega$.

We therefore write the boundary condition as $f(\mathbf{x}) = g(\mathbf{x})$ on $\partial\Omega$.

Putting this together, we get the following problem:

$$\begin{cases} \frac{\partial f}{\partial t} = \Delta f \\ f = g \text{ on } \partial\Omega \end{cases}$$

Then if we want to find the steady-state solution, we can set $\frac{\partial f}{\partial t} = 0$:

$$\begin{cases} 0 = \Delta f \\ f = g \text{ on } \partial\Omega \end{cases}$$

$\Delta f = \nabla^2 f = 0$ is called the **Laplace equation**.

$$\begin{cases} 0 = \Delta f \\ f = g \text{ on } \partial\Omega \end{cases}$$

We can view the Laplace equation as an optimization problem!

We just need to find a function f that *minimizes* the Laplacian over the region Ω .

(We'll want to integrate this over Ω to get a scalar value.)

f can be any function that's defined over Ω , as long as:

1. the boundary condition is satisfied, and
2. the Laplacian of f exists: f should be twice differentiable everywhere in Ω :
 $f \in C^2(\Omega)$

Putting this all together, in $\mathbb{R}^3$, this looks like

$$\begin{aligned} &\text{minimize} \quad \frac{1}{2} \int_{\Omega} \|\nabla f(x, y, z)\|^2 dx dy dz \\ &\text{subject to} \quad f = g \text{ on } \partial\Omega, f \in C^2(\Omega) \end{aligned}$$

Using optimization to solve PDEs in this way is called the **Dirichlet principle**.

Example: Stock portfolio optimization

Suppose you have m dollars to invest in n different stocks.

Let x_i be the number of dollars you invest in stock i . (You can have a *long position* $x_i > 0$ or a *short position* $x_i < 0$.)

Let p_i be the relative price change of stock i over some time period.

Then we can write

$$\mathbf{p} = \begin{bmatrix} p_1 \\ \vdots \\ p_n \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}.$$

The overall return on your portfolio is simply the sum of the returns on the individual stocks, which we can write as

$$\sum_{i=1}^n p_i x_i = \mathbf{p}^T \mathbf{x}$$

But suppose we don't know ahead of time what the returns $\mathbf{p}$ will be.

We might model them as random variables, with some mean $\mu \in \mathbb{R}^n$ and covariance matrix $\Sigma \in \mathbb{R}^{n \times n}$.

$\Sigma \in \mathbb{R}^{n \times n}$ captures not only how much each stock moves, but also how much pairs of stocks move *together*.

Then our **profit**, or expected return on our portfolio, is

$$\mathbb{E}[\mathbf{p}^T \mathbf{x}] = \mathbf{1}^T \mathbf{x}$$

And our **risk**, or the variance of the returns, is

$$\text{Var}[\mathbf{p}^T \mathbf{x}] = \mathbf{x}^T \Sigma \mathbf{x}$$

OK. So suppose we have m dollars to invest, we want to earn at least a certain profit r , and we want to minimize our risk. We also don't want to short any stocks.

We can write this as the following optimization problem:

$$\begin{array}{ll} \text{minimize} & \mathbf{x}^\top \Sigma \mathbf{x} \\ \text{subject to} & \begin{cases} \mu^\top \mathbf{x} \geq r & (\text{minimal acceptable return}) \\ \mathbf{1}^\top \mathbf{x} = m & (\text{finite money}) \\ \mathbf{x} \geq 0 & (\text{no short positions}) \end{cases} \end{array}$$

This type of problem is called a **quadratic programming problem**, because the objective function is quadratic:

$$\mathbf{x}^\top \Sigma \mathbf{x} = \sum_{i=1}^n \sum_{j=1}^n \sigma_{ij} x_i x_j$$

A side note on the word “programming”

We will often use the words “program” or “programming” to refer to the process of solving optimization problems.

This usage isn't from computer science!

It actually comes from the fact that much of this mathematics was developed in WWII, to solve problems in *planning* or *scheduling*.

But in general, if you see something like “quadratic programming”, that means “doing optimization with a quadratic objective function”.

**Other places where optimization is
used**

Terms that imply optimization

- maximum likelihood
 - in statistics: find parameters that maximize the likelihood of observed data
- minimum entropy
 - in information theory: find distributions that minimize uncertainty
- least action
 - in physics: find paths that minimize the energy expended by a system
- shortest path
 - in graph theory: find the path of least distance between two points
- anything involving maximum/minimum, highest/lowest, shortest/longest, etc.

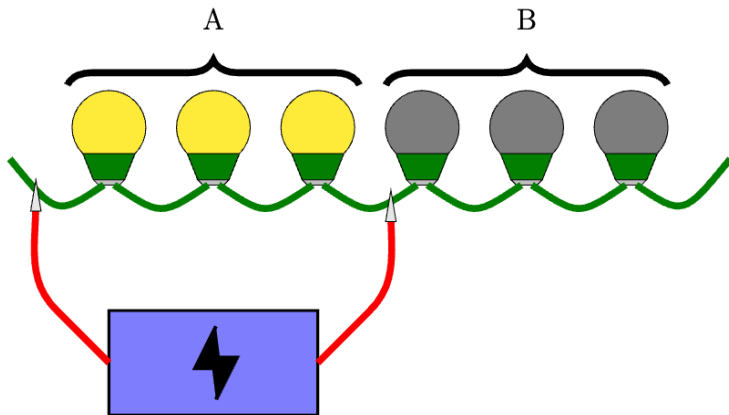
**The simplest optimization
algorithm in this course**

Method of bisection

Next lecture, we will use the method of bisection to find the root of a univariate continuous function $f(x)$ – the value x where $f(x) = 0$.

But the method itself is very simple!

Suppose you have a set of Christmas lights to hang, but they don't work. One bulb is dead, but which one? Too slow to test each one.



Or suppose you know that *somebody* in a group of 1024 has measles.

If you test each person individually, you need 1024 tests.

Instead, you can divide the group into two groups of 512.

Mix the tissue samples in each group and test each group.

Then take the group that tested positive and divide it in half again, and repeat the process.

You'll find the person with measles in just 10 tests.

The method of bisection reduces the testing complexity from linear ($O(n)$) to logarithmic ($O(\log_2 n)$).

This is an exponential improvement!

Throughout this course, we will see how leveraging mathematical insights can lead to much more efficient algorithms.

Calculus for real-life problems

In your calculus classes, you probably learned how to differentiate lots of weird made-up functions for practice, like:

$$f(x, y, z) = 2x^3 \sin 4y - 5e^{6xy} + \frac{x^7}{z^8 + 9}$$

Or perhaps you looked at weird functions to illustrate various pathologies, like

$$f(x) = \begin{cases} \frac{1}{q} & x = \frac{p}{q} \in \mathbb{Q}, \gcd(p, q) = 1, \\ 0 & x \in \mathbb{R} \setminus \mathbb{Q}, \end{cases}$$

$$f(x) = \begin{cases} \frac{1}{q} & x = \frac{p}{q} \in \mathbb{Q}, \gcd(p, q) = 1, \\ 0 & x \in \mathbb{R} \setminus \mathbb{Q}, \end{cases}$$

(This function equals the fraction $1/q$ if x is rational, and 0 if x is irrational. It's called the **Popcorn function**.)

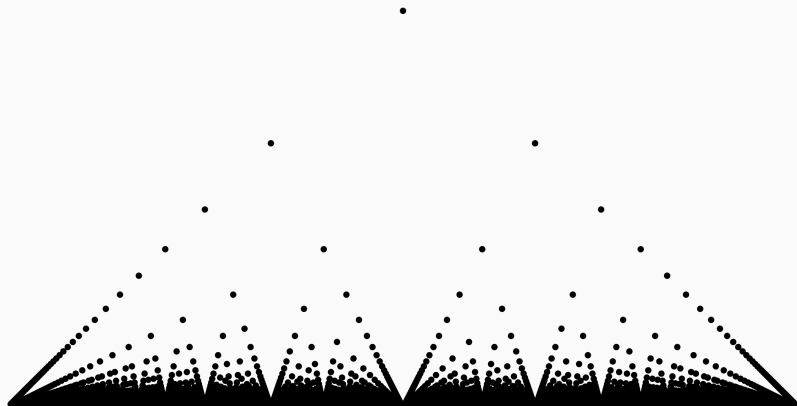


Figure 2: Popcorn function

But in real life, you'll never see functions like this!

In science and engineering, you are much more likely to see functions like these, where $\mathbf{x}$ is an n -dimensional vector:

$$f(\mathbf{x}) = \sqrt{\mathbf{x}^\top \Sigma \mathbf{x}}, \quad g(\mathbf{x}) = \exp \left[-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right], \quad h(\mathbf{x}) = \log (e^{x_1} + \cdots + e^{x_n})$$

or these, where Σ is an $n \times n$ matrix:

$$F(\Sigma) = \sqrt{\mathbf{x}^\top \Sigma \mathbf{x}}, \quad G(\Sigma) = \exp \left[-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right], \quad H(\Sigma) = \log \det(\Sigma)$$

These functions all come from real-life problems:

- f and F from the Mahalanobis distance;
- g and G from the Gaussian density;
- h and H from geometric programming and semidefinite programming respectively.

h , for example, is the log-sum-exp or “LSE” function. Its derivative is called the **softmax** function, which is important in machine learning.

$$f(\mathbf{x}) = \sqrt{\mathbf{x}^\top \Sigma \mathbf{x}}, \quad g(\mathbf{x}) = \exp \left[-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right], \quad h(\mathbf{x}) = \log (e^{x_1} + \dots + e^{x_n})$$

$$F(\Sigma) = \sqrt{\mathbf{x}^\top \Sigma \mathbf{x}}, \quad G(\Sigma) = \exp \left[-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right], \quad H(\Sigma) = \log \det(\Sigma)$$

Can you find the first and second derivatives of these functions?

It's not easy. To find the second derivative of H , you'll need to differentiate $\varphi(X) = X^{-1}$, the function that takes a matrix X and returns its inverse.

Can you even take the derivative of the square of a matrix, $\psi(X) = X^2$?

Example: Electronic circuit design

We want to choose the widths and lengths $x_1, \dots, x_n$ of various devices and wires connecting them in an electronic circuit.

(To get a sense of scale: A16 bionic chip made by TSMC for use in the iPhone 14 Pro and Pro Max has 16 billion such devices and a similar number of wires connecting them!)

The constraints $g_1, \dots, g_m, h_1, \dots, h_p$ are:

- limits on sizes of components (we can only make transistors so small)
- timing requirements (the circuit must operate at a certain speed)
- limits on the total area of the circuit (the chip needs to fit on the phone)

The objective function f is the total power consumption of the circuit.

So our optimization problem is: find the widths and lengths $x_1^*, \dots, x_n^*$ that minimize the total power consumption of the circuit, subject to the manufacturing constraints.

We can formulate this as a *geometric programming* problem: $f, g_1, \dots, g_m, h_1, \dots, h_p$ all take the form

$$LSE(A\mathbf{x} + \mathbf{b})$$

where

$$LSE(\mathbf{x}) := \log(e^{x_1} + \dots + e^{x_n})$$

Types of optimization problems

Reminder: the general form of an optimization problem is

$$\begin{aligned} & \text{minimize } f(x) \\ & \text{subject to } g_i(x) \leq 0, \quad i = 1, \dots, m \\ & \quad \quad \quad h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

Depending on the specific forms of f, g_i, h_j , we can classify optimization problems into different types, ranging from simple (and easy to solve!) to complex (and harder to solve).

These range from:

- Univariate optimization
- Unconstrained optimization
- Linear programming
- Least-squares problems
- Quadratic programming
- Second-order cone programming
- Semidefinite programming
- Geometric programming
- Convex optimization

$$n = 1$$

$$\begin{array}{ll}\text{minimize} & f(x) \\ \text{subject to} & a \leq x \leq b\end{array}$$

$m = 0, p = 0$ (there are zero equality constraints and zero inequality constraints)

minimize $f(\mathbf{x})$

where $\mathbf{x}$ runs over all possible values in the domain of f .

Side note: why we care about univariate and unconstrained optimization

- Simple to solve
- Techniques we develop can be extended to multivariate and constrained problems
- Often we may reduce multivariate and constrained problems to sequences of univariate and unconstrained problems

Moving along: Linear programming (LP)

f, g_i, h_i are all linear functions

$$\begin{aligned} & \text{minimize } c_1x_1 + \cdots + c_nx_n \\ & \text{subject to } a_{i1}x_1 + \cdots + a_{im}x_m \leq b_i, \quad i = 1, \dots, m \end{aligned}$$

Why did I only write the inequality constraints?

We can always write an equality constraint $h_j(x) = 0$ as two inequality constraints: $h_j(x) \leq 0$ and $-h_j(x) \leq 0$.

We can write the general form of an LP as:

$$\begin{aligned} & \text{minimize } \mathbf{c}^\top \mathbf{x} \\ & \text{subject to } A\mathbf{x} \leq \mathbf{b}. \end{aligned}$$

Here f is a sum of squares of linear functions:

$$f(\mathbf{x}) = \sum_{i=1}^n (a_{i1}x_1 + \cdots + a_{in}x_n - b_i)^2 = \|A\mathbf{x} - \mathbf{b}\|_2^2$$

Then the problem is:

$$\text{minimize} \quad \|A\mathbf{x} - \mathbf{b}\|_2^2$$

We commonly use LS when solving linear regression problems.

Quadratic programming allows quadratic terms in the objective and constraint functions.

Given matrices $A_i \in \mathbb{R}^{n \times n}$ and vectors $b_i \in \mathbb{R}^n$, for $i = 0, 1, \dots, p$, the general form of a QP is:

$$\begin{aligned} & \text{minimize} && \mathbf{x}^\top A_0 \mathbf{x} + \mathbf{b}_0^\top \mathbf{x} + c_0 \\ & \text{subject to} && \mathbf{x}^\top A_i \mathbf{x} + \mathbf{b}_i^\top \mathbf{x} + c_i \leq 0, \quad i = 1, \dots, p \end{aligned}$$

Quadratic programming is particularly useful for problems involving distances, energies, or other variances (as in our portfolio optimization example).

Second-order cone programming

The constraint in a second-order cone program (SOCP) is geometrically like requiring points $\mathbf{x}$ to lie on or inside an ice-cream cone shape in space:

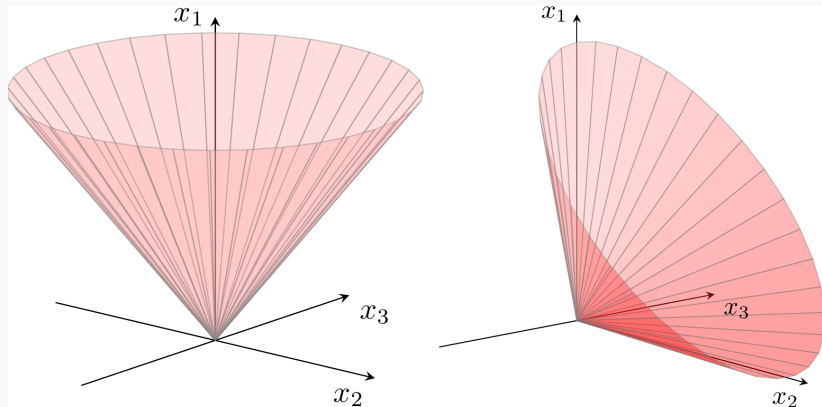


Figure 3: Cone

We can write a second-order cone constraint as:

$$\|A_i \mathbf{x} + \mathbf{b}_i\|_2 \leq \mathbf{c}_i^T \mathbf{x} + d_i$$

So the optimization problem is:

$$\begin{array}{ll} \text{minimize} & f(\mathbf{x}) \\ \text{subject to} & \|A_i \mathbf{x} + \mathbf{b}_i\|_2 \leq \mathbf{c}_i^T \mathbf{x} + d_i, \quad i = 1, \dots, p \end{array}$$

A semidefinite program (SDP) is a type of optimization problem that involves a matrix variable X and a linear objective function.

The general form of an SDP is:

For $A_i \in \mathbb{S}^n, b_i \in \mathbb{R}$,

$$\begin{aligned} & \text{minimize} && \text{tr}(A_0 X) \\ & \text{subject to} && \text{tr}(A_i X) \leq b_i, \quad i = 1, \dots, p, \\ & && X \in \mathbb{S}_+^n \end{aligned}$$

Given $A_i \in \mathbb{R}^{m_i \times n}$ $\mathbf{b}_i \in \mathbb{R}^{m_i}$, the general form of a geometric program is:

$$\begin{array}{ll}\text{minimize} & \text{LSE}(A_0 \mathbf{x} + \mathbf{b}_0) \\ \text{subject to} & \text{LSE}(A_i \mathbf{x} + \mathbf{b}_i) \leq 0, \quad i = 1, \dots, p\end{array}$$

Remember that $\text{LSE}(\mathbf{x}) = \log(\sum_{i=1}^n e^{x_i})$.

Geometric programming is useful for problems involving ratios, products, and other non-linear functions.

A convex optimization problem is one where the objective and constraint functions are all *convex*.

Convex functions have the property that the line segment between any two points on the function lies above the function. There are no “holes” or “valleys” in the function where optimization might get stuck.

This means that the function is *concave up*, and that any local minimum is a global minimum.

LP, LS, SOCP, SDP, and GP are all special cases of convex optimization problems.

- Polynomial optimization:
 - f, g_i, h_i are all multivariate polynomial functions.
- Nonsmooth optimization.
 - f, g_i, h_i may not be differentiable, e.g., involving $\|\mathbf{x}\|_1 = |x_1| + \dots + |x_n|$ or $\|\mathbf{x}\|_\infty = \max\{|x_1|, \dots, |x_n|\}$
- Vector or multiobjective optimization.
 - f, g_i, h_i are vector-valued functions, i.e., $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ where $m > 1$.

- Integer programming.
 - Variables are all integers or discrete, i.e., $\mathbf{x} \in \{\pm 1\}^n$ or $\mathbf{x} \in \mathbb{Z}^n$ instead of $\mathbf{x} \in \mathbb{R}^n$.
- Mixed integer programming.
 - Some variables continuous, some discrete, e.g., $\mathbf{x} = (x_1, x_2, x_3, x_4) \in \mathbb{R} \times [0, \infty) \times \mathbb{Z} \times \{0, 1\}$
- Combinatorial optimization.
 - Discrete variables and constraints given by combinatorial structures, e.g., graphs, hypergraphs, combinatorial designs, matroids.

- Dynamic programming.
 - f, g_i, h_i are functions of discrete paths.
- Calculus of variations.
 - f, g_i, h_i are functions of functions.
- Optimal control.
 - g_i, h_i are first-order ordinary differential equations.
- PDE-constrained optimization.
 - g_i, h_i are partial differential equations.
- Infinite-dimensional optimization.
 - f, g_i, h_i are functions on infinite-dimensional spaces.

Course roadmap

We'll follow this route:

1. Univariate and unconstrained optimization
 - Most intuitive and simplest
2. Multivariate equations
 - Same fundamental principles, but more complex
3. Multivariate constrained optimization
4. Multivariate unconstrained optimization
5. General convex optimization

Let's begin!

Univariate optimization

We'll start with optimizing real-valued functions of one real variable, defined on a subset $X \subset \mathbb{R}$:

$$f : X \rightarrow \mathbb{R}$$

X is the domain of f .

Typically, $X = [a, b]$ or (a, b)

Remember:

$X = [a, b]$ is the set of all real numbers between a and b , *including* a and b . We call this a **closed interval**.

$X = (a, b)$ is the set of all real numbers between a and b , *excluding* a and b . We call this an **open interval**.

A classification of functions

To prove that various optimization algorithms converge, and how quickly, we will need to understand properties of the functions f .

How well-behaved is f ? Is it continuous? Differentiable? Twice differentiable? Etc.

Here, we'll consider four types of functions:

1. Continuous
2. Lipschitz continuous
3. Continuously differentiable
4. Twice continuously differentiable

A continuous function is one that has no *jumps* or *breaks* in its graph.

This property is going to be very important because it will allow the *extreme value theorem* (will talk about soon!), which says that a continuous function on a closed and bounded interval attains its maximum and minimum values.

That means that if we are looking for a minimum or maximum (on a closed and bounded interval), we will know that it exists.

If we have a function $f : X \rightarrow \mathbb{R}$ and $X = (a, b)$ (remember: endpoints excluded), f is continuous if:

1. $\lim_{x \rightarrow x_0} f(x) = f(x_0)$ for all $x_0 \in (a, b)$

For the case where $X = [a, b]$, we need to update the definition a little so it will work at the endpoints.

f is continuous if:

$\lim_{x \rightarrow x_0} f(x) = f(x_0)$ for all $x_0 \in (a, b)$,

and

$$\lim_{x \rightarrow a^+} f(x) = f(a),$$

$$\lim_{x \rightarrow b^-} f(x) = f(b)$$

That is, f is continuous on (a, b) , right continuous at a , and left continuous at b .

$C^0(X)$ (“C-zero of X ”) is the set of all continuous functions on X .

We write $C^0(X) = \{f : X \rightarrow \mathbb{R} \mid f \text{ is continuous on } X\}$

Give the $\varepsilon - \delta$ definitions for continuity of f at x_0 for the cases where $X = (a, b)$ and $X = [a, b]$.

Lipschitz continuous functions are continuous and also *well-behaved* in the sense that the function values don't change too much from one point to another.

Geometrically, there will be a maximum slope for the function's graph (determined by the Lipschitz constant γ)

Will be important for analyzing how quickly algorithms converge.

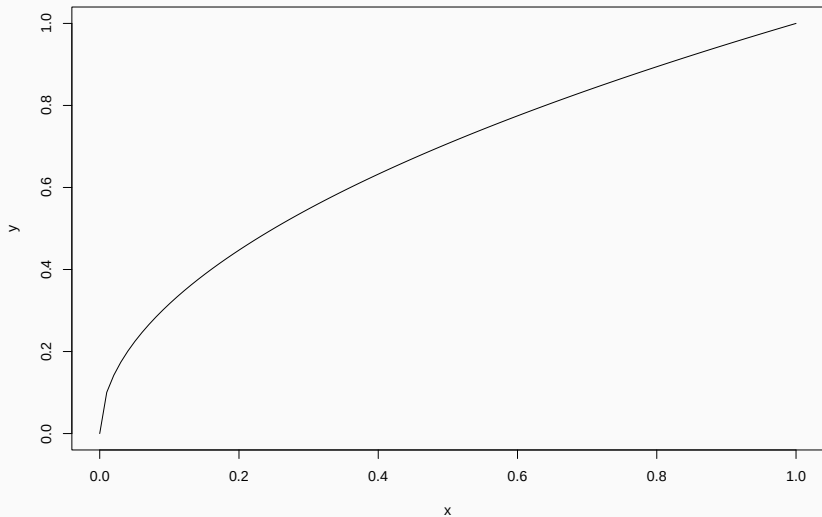
A function $f : X \rightarrow \mathbb{R}$ is Lipschitz continuous if there exists a constant $\gamma \geq 0$ such that:

$$|f(x) - f(y)| \leq \gamma |x - y|$$

for all $x, y \in X$.

Functions can be continuous but not Lipschitz

The function $\sqrt{x}$ near 0 has an unbounded derivative:



We can't find a constant γ so that the slope of the function is always less than γ .

$Lip_\gamma(X)$ ("Lip-gamma of X") is the set of all functions that are Lipschitz continuous with constant γ .

We write $Lip_\gamma(X) = \{f : X \rightarrow \mathbb{R} \mid f \text{ is Lipschitz with constant } \geq \gamma\}$

A function is continuously differentiable if it is differentiable and its derivative doesn't have any breaks or jumps.

The smoothness of the derivative will allow us to use derivative-based optimization methods, knowing that e.g. moving in the direction of steepest descent will lead us toward a minimum.

Definition of continuous differentiability

A function $f : X \rightarrow \mathbb{R}$ is continuously differentiable if

$$f'(x) := \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

exists for all $x \in X$ and $f' : X \rightarrow \mathbb{R}$ is continuous.

Definition of continuous differentiability (cont'd)

If $X = [a, b]$, then we need derivatives to exist at the end points. These will be defined as left and right derivatives:

$$f'(a^+) := \lim_{h \rightarrow 0^+} \frac{f(a+h) - f(a)}{h}, \quad f'(b^-) := \lim_{h \rightarrow 0^-} \frac{f(b+h) - f(b)}{h},$$

$C^1(X)$ (“C-one of X”) is the set of all functions that are continuously differentiable on X .

We write $C^1(X) = \{f : X \rightarrow \mathbb{R} \mid f \text{ is continuously differentiable on } X\}$

Twice continuous differentiability means that the function's second derivative exists and is continuous.

This will allow us to use the curvature of the function to help us more efficiently find the minimum.

Definition of twice continuous differentiability

A function $f : X \rightarrow \mathbb{R}$ is twice continuously differentiable if

$$f''(x) := \lim_{h \rightarrow 0} \frac{f'(x+h) - f'(x)}{h}$$

exists for all $x \in X$ and $f'' : X \rightarrow \mathbb{R}$ is continuous.

How would you modify this definition at the end points if $X = [a, b]$?

$C^2(X)$ ("C-two of X") is the set of all functions that are twice continuously differentiable on X .

We write $C^2(X) = \{f : X \rightarrow \mathbb{R} \mid f \text{ is twice continuously differentiable on } X\}$

Properties of continuous functions

What properties do continuous functions have that are important for optimization?

- Boundedness
- Extreme value theorem

On a closed interval, a continuous function is bounded.

Why? Basically, if the function is not bounded, it will go to ∞ or $-\infty$ somewhere. Close to these points, the function will be changing infinitely fast, and this will break our definition of continuity.

Lemma

If $f : [a, b] \rightarrow \mathbb{R}$ is continuous, then f is bounded on $[a, b]$.

Before we go on, a reminder from Real Analysis:

The **supremum** of a set S is the least upper bound of S .

That means that the supremum is the smallest number that is greater than or equal to all the numbers in the set. If the function actually attains its supremum, then it is a maximum.

The **infimum** of a set S is the greatest lower bound of S .

It's the largest number that is less than or equal to all the numbers in the set. If the function actually attains its infimum, then it is a minimum.

The extreme value theorem says that a continuous function on a closed and bounded interval attains its supremum and infimum.

That means that if we are looking for a minimum or maximum (on a closed and bounded interval), we will know that it exists.

Theorem

If $f : [a, b] \rightarrow \mathbb{R}$ is continuous and $M = \sup_{x \in [a, b]} f(x)$ and $m = \inf_{x \in [a, b]} f(x)$, then there exist points $x_1, x_2 \in [a, b]$ such that $f(x_1) = m$ and $f(x_2) = M$.

Proof of the extreme value theorem

We will prove this with a contradiction: we'll construct a function that would violate the boundedness property if the supremum is not attained.

Suppose the extreme value theorem is false - that is, suppose f is continuous on $[a, b]$ and has supremum M , but it never attains its supremum - that is, $f(x) < M$ for all $x \in [a, b]$.

Now let's define a new function $g : [a, b] \rightarrow \mathbb{R}$ given by

$$g(x) = \frac{1}{M - f(x)}$$

This function is continuous on $[a, b]$ (because $M - f(x)$ is never 0).

But from Lemma 14.1, g is bounded. So there exists some $K > 0$ such that for all $x \in [a, b]$,

$$g(x) \leq K \quad \Rightarrow \quad \frac{1}{M - f(x)} \leq K \quad \Rightarrow \quad f(x) \leq M - \frac{1}{K}$$

But this means that $f(x) \leq M - \frac{1}{K}$ for all $x \in [a, b]$ - so M cannot be a supremum. This is a contradiction, so we conclude that in fact f attains its supremum.

The extreme value theorem and boundedness don't hold for open intervals.

For example, $f(x) = x^2$ on $(0, 2)$

The supremum of f is 4, and the infimum is 0.

But these values are never attained, since $x = 0$ and $x = 2$ are not in the domain.

Another example:

$$f(x) = \tan^{-1} x \text{ on } (-\infty, \infty)$$

The supremum is $M = \pi/2$ and the infimum is $m = -\pi/2$.

But these values are never actually attained, since x never actually equals $\pm\infty$.

Properties of differentiable functions

These definitions apply to any real-valued function (not necessarily differentiable or continuous):

Definition

Let $f : X \rightarrow \mathbb{R}$ be any function and $x_0 \in X$. Then

$f(x_0)$ is a **local minimum** if $f(x) \geq f(x_0)$ for all x in some neighborhood of x_0 ;

x_0 is a **local minimizer** if $f(x_0)$ is a local minimum.

$f(x_0)$ is a **local maximum** if $f(x) \leq f(x_0)$ for all x in some neighborhood of x_0 ;

x_0 is a **local maximizer** if $f(x_0)$ is a local maximum.

Let $f : X \rightarrow \mathbb{R}$ be any function and $x_0 \in X$. Then

$f(x_0)$ is a **global minimum** if $f(x_0) = \inf_{x \in X} f(x)$;

x_0 is a **global minimizer**.

$f(x_0)$ is a **global maximum** if $f(x_0) = \sup_{x \in X} f(x)$;

x_0 is a **global maximizer**.

Theorem

If x_0 is a local optimizer of f , then either $f'(x_0) = 0$ or $f'(x_0)$ does not exist.

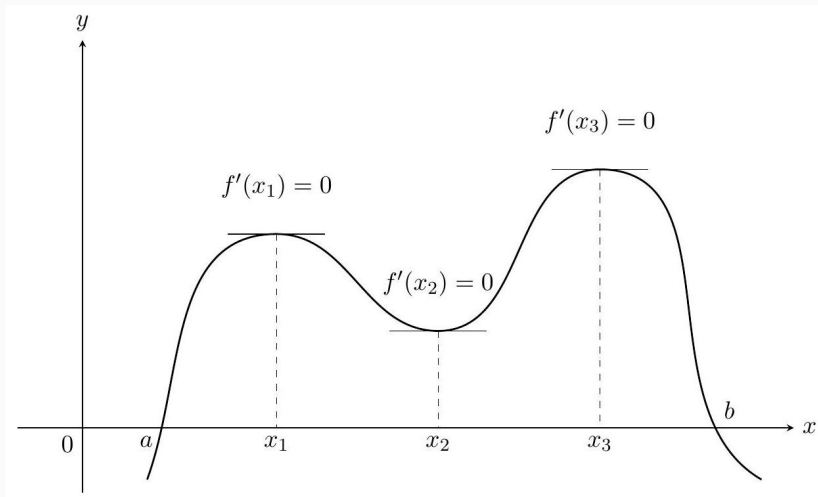


Figure 4: $X = [a, b]$; x_1, x_2, x_3 are stationary points; x_1, x_3 are local maximizers; x_2 is local minimizer; x_3 is global maximizer; a, b are global minimizers.

Proof:

Intuitively, if the derivative were positive, moving slightly left would decrease the function; if negative, moving slightly right would decrease the function.

However, in a small area around a local minimizer, a move in *any* direction must either increase the function or leave it unchanged.

So the right derivative must be greater than or equal to 0, and the left derivative must be less than or equal to 0.

Hence the derivative must be 0.

Suppose x_0 is a local minimizer. There exists $\delta > 0$ such that for all $x \in B_\delta(x_0) := \{x \in \mathbb{R} : |x - x_0| < \delta\}$, we have $f(x) \geq f(x_0)$. Hence

$$\frac{f(x) - f(x_0)}{x - x_0} \begin{cases} \geq 0 & \text{if } x_0 < x < x_0 + \delta \\ \leq 0 & \text{if } x_0 - \delta < x < x_0 \end{cases}$$

If $f'(x_0)$ exists, then

$$f'(x_0) = \lim_{x \rightarrow x_0^+} \frac{f(x) - f(x_0)}{x - x_0} \geq 0, \quad (\text{right limit})$$

$$f'(x_0) = \lim_{x \rightarrow x_0^-} \frac{f(x) - f(x_0)}{x - x_0} \leq 0. \quad (\text{left limit})$$

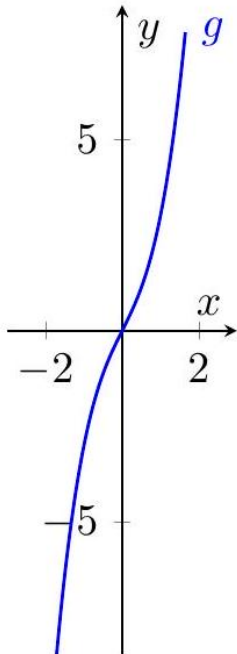
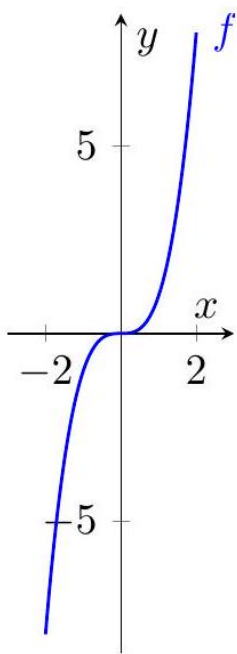
Hence $f'(x_0) = 0$.

Definition

If $f'(x_0) = 0$, then x_0 is called a **stationary point** or **critical point** and $f(x_0)$ is called a **stationary value** or **critical value**.

By Theorem 15.1, if $f \in C^1(X)$, then every local optimizer (minimizer or maximizer) must be a stationary point.

But the converse is not true. We can have stationary points that are not minimizers or maximizers.



y h

Example: ReLU function

The ReLU (**R**ectified **L**inear **U**nit) function is an important function in machine learning: $f(x) = \max(x, 0)$

Its local minimum occurs at a non-differentiable point!

